Laboratorio 01 - Mortalidad

Lenin Pavón

2026-04-21

Introducción

Para poder realizar nuestro código es menester cargar las siguientes bibliotecas. De particular interés es `readxl` que nos permitirá leer directamente los archivos en xlsx.

- readxl
- dplyr
- tidyverse
- forecast
- ggplot2

Antes de continuar, quisiera comentar que el proceso de creación del pdf masacró de manera inexorable la belleza de algunas gráficas. Le recomiendo a la lectora consultar cada gráfica accediendo al directorio `plots`. Están (en su mayoría) en el mismo orden en el que se presentan en el reporte de laboratorio.

Cargamos los datos con `Pob_mitad` las proyecciones a mitad de año y `TEM` el dataframe de las defunciones

```
Pob_mitad   <- read_excel("data/0_Pob_Mitad_1950_2070.xlsx")
Defunciones <- read_excel("data/1_Defunciones_1950_2070.xlsx")
TEM <- Defunciones
```

Tasas específicas de mortalidad

Calculamos las tasas específicas de mortalidad por edad, sexo y año

```
TEM$POBLACION <- Pob_mitad$POBLACION
TEM$TEM       <- TEM$DEFUNCIONES/TEM$POBLACION
```

Nos interesa realizar el cálculo a nivel nacional así que retiraremos los datos estatales

```
TEM <- TEM %>% filter(ENTIDAD == "República Mexicana")
TEM <- TEM %>% select(-ENTIDAD)
```

Para el cálculo de los datos de proyección (2020-2070), requerimos de un lugar donde almacenarlo que llamaremos `Base_CD_TEM`

```
Base_CD_TEM <- matrix(NA, 220, 70)
colnames(Base_CD_TEM) = seq(1950,2019)
```

Aquí almacenaremos las TEM extraídas con anterioridad con base en el año y sexo

```
for (i in 1950:2019)
{
  for (j in 1:220)
  {
    if(j <= 110)
```

```r
    { # Hombres
      valor <- TEM %>%
        filter(SEXO == "Hombres", AÑO == i, EDAD == (j-1)) %>%
        pull(TEM) # Extraer el valor
    } else { # Mujeres
      valor <- TEM %>%
        filter(SEXO == "Mujeres", AÑO == i, EDAD == (j-111)) %>%
        pull(TEM) # Extraer el valor
    }
    # Asignamos el valor a la matriz
    Base_CD_TEM[j, i-1949] = ifelse(length(valor) > 0, valor, NA)
  }
}
```

Una vez realizado este trabajo, obligamos a la transmutación alquímica de la matriz a un dataframe con el mismo nombre

```r
Base_CD_TEM <- as.data.frame.matrix(Base_CD_TEM)
```

Sin embargo, este reetiquetado requiere de cierto andamiaje para funcionar apropiadamente, particularmente con respecto a los índices del dataframe. Definimos entonces a los vectores auxiliares

```r
age   <-    0:109
years <- 1950:2019
sex   <- rep(c("Hombre, Mujer"), each = 110)
```

Así, definimos al dataframe en formato largo con las tasas específicas de mortalidad `df`

```r
df <- as.data.frame(Base_CD_TEM)
colnames(df) <- as.character(years)
df$Edad <- rep(age, 2)
df$Sexo <- sex
```

Ahora hacemos compatible nuestro dataframe con ggplot convirtiéndolo a un formato long

```r
df_largo <- df %>%
  pivot_longer(cols = starts_with("19") | starts_with("20"),
               names_to = "Año",
               values_to = "Tasa") %>%
  mutate(Año = as.integer(Año))
```

Graficando

```r
ggplot(df_largo, aes(x = Edad, y = Tasa, color = Sexo)) +
  geom_line() +
  facet_wrap(~Año, ncol = 7) +
  theme_minimal() +
  labs(title = "Tasas específicas de mortalidad por edad, sexo y año",
       x = "Edad",
       y = "Tasa de mortalidad",
       color = "Sexo")
```

```
## Warning: Removed 110 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

Corrigiendo las Tasas Específicas de Mortalidad

Para llevar a cabo las correcciones deseadas se vuelve necesario tener a la mano una tabla con la esperanza de vida `ev` y la tabla modelo de las Naciones Unidas `tabla_modelo`.

```
ev           <- read_excel("data/EV_1950_2019.xlsx")
tabla_modelo <- read_excel("data/MLT_UN2011_130_1y_complete.xlsx")
```

De aquí tomamos la esperanza de vida de cada año, filtramos la tabla modelo por el patrón *Latin*, el sexo, y la esperanza de vida `e0`. Posteriormente filtramos las edades extremas (85+) e imputamos los valores correspondientes

```
imputar_mx_edades_extremas <- function(tabla_modelo, esperanza_vida, mxt, años = 1950:2019) {
  for (año in años) {
    for (sexo in c("Male", "Female")) {

      # Extraer esperanza de vida como entero
      sexo_spa <- if( sexo == "Male" ) "Hombres" else "Mujeres"
      e0 <- as.numeric(ev[which(ev$año == año & ev$sexo == sexo_spa), ]$EV)

      # Filtrar tabla modelo por patrón Latin, sexo y E0 exacto
      tabla_filtrada <- tabla_modelo[
        tabla_modelo$Family == "Latin" &
          tabla_modelo$Sex == sexo &
          tabla_modelo$E0 == e0, ]

      # Extraer mx1 para edades 90-109
      edades_extremas <- 85:109
```

```r
      mx_vals <- tabla_filtrada[tabla_filtrada$age %in% edades_extremas, "mx1"]

      # Determinar filas en mxt
      filas_mxt <- if (sexo == "Male") edades_extremas + 1 else edades_extremas + 111

      # 6. Determinamos columna en mxt
      col_mxt <- which(colnames(mxt) == año)

      # 7. Imputar valores
      mxt[filas_mxt, col_mxt] <- mx_vals
    }
  }
  return(mxt)
}
```

Aplicando este método a nuestros datos,

```r
ev[,3] = round(ev[,3])
mxt = imputar_mx_edades_extremas(tabla_modelo, ev, Base_CD_TEM, años = 1950:2019)
mxt = as.data.frame.matrix(mxt)
```

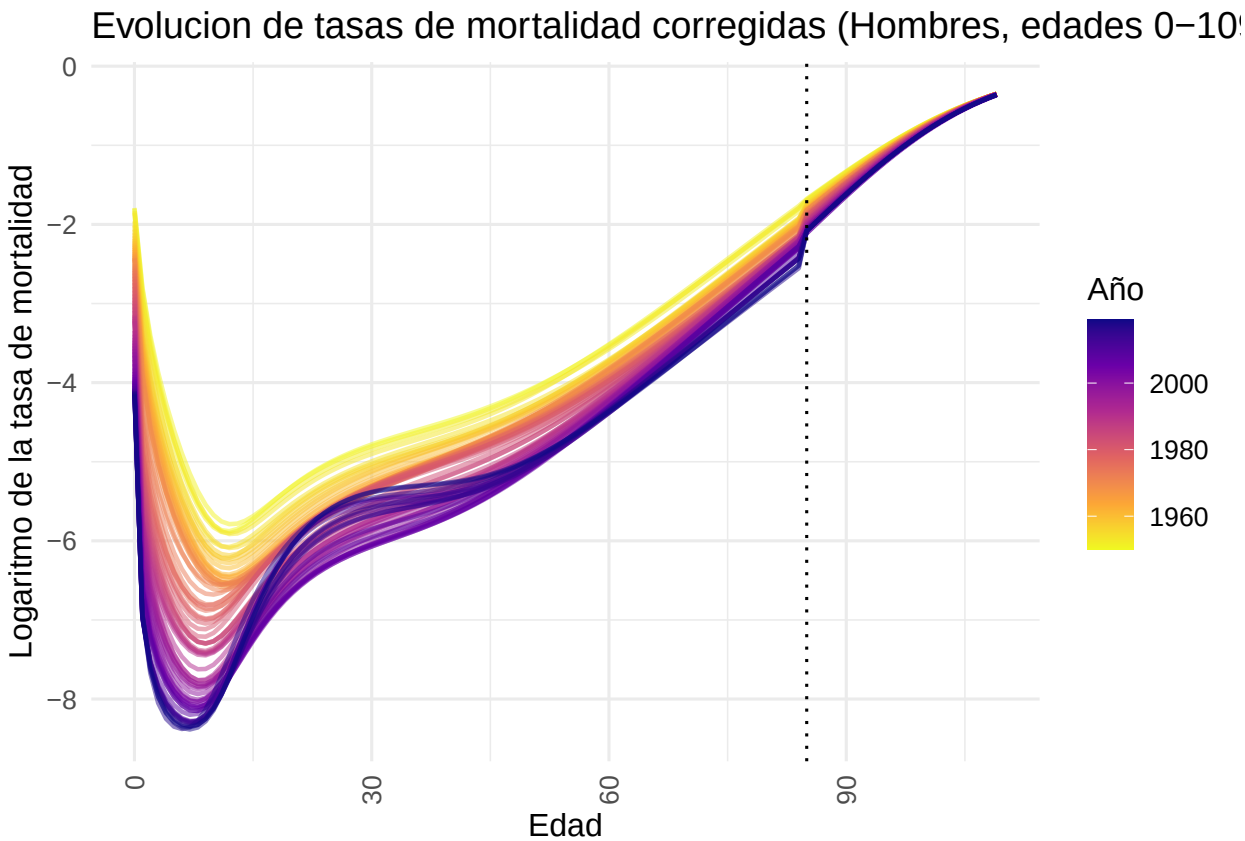
Realizando el mismo andamiaje para las etiquetas del dataframe

```r
sex_eng <- rep(c("Male", "Female"), each = 110)
# Convertir mxt a data frame largo
df <- as.data.frame(mxt)
colnames(df) <- as.character(years)
df$Edad <- rep(age, 2)
df$Sexo <- sex_eng

# Reorganizar en formato largo
df_largo <- df %>%
  pivot_longer(cols = starts_with("19") | starts_with("20"),
               names_to = "Año",
               values_to = "Tasa") %>%
  mutate(Año = as.integer(Año))

# Filtrar para hombres y todas las edades
df_hombres <- df_largo %>%
  filter(Sexo == "Male", Año >= 1950, Año <= 2019)
df_mujeres <- df_largo %>%
  filter(Sexo == "Female", Año >= 1950, Año <= 2019)
```

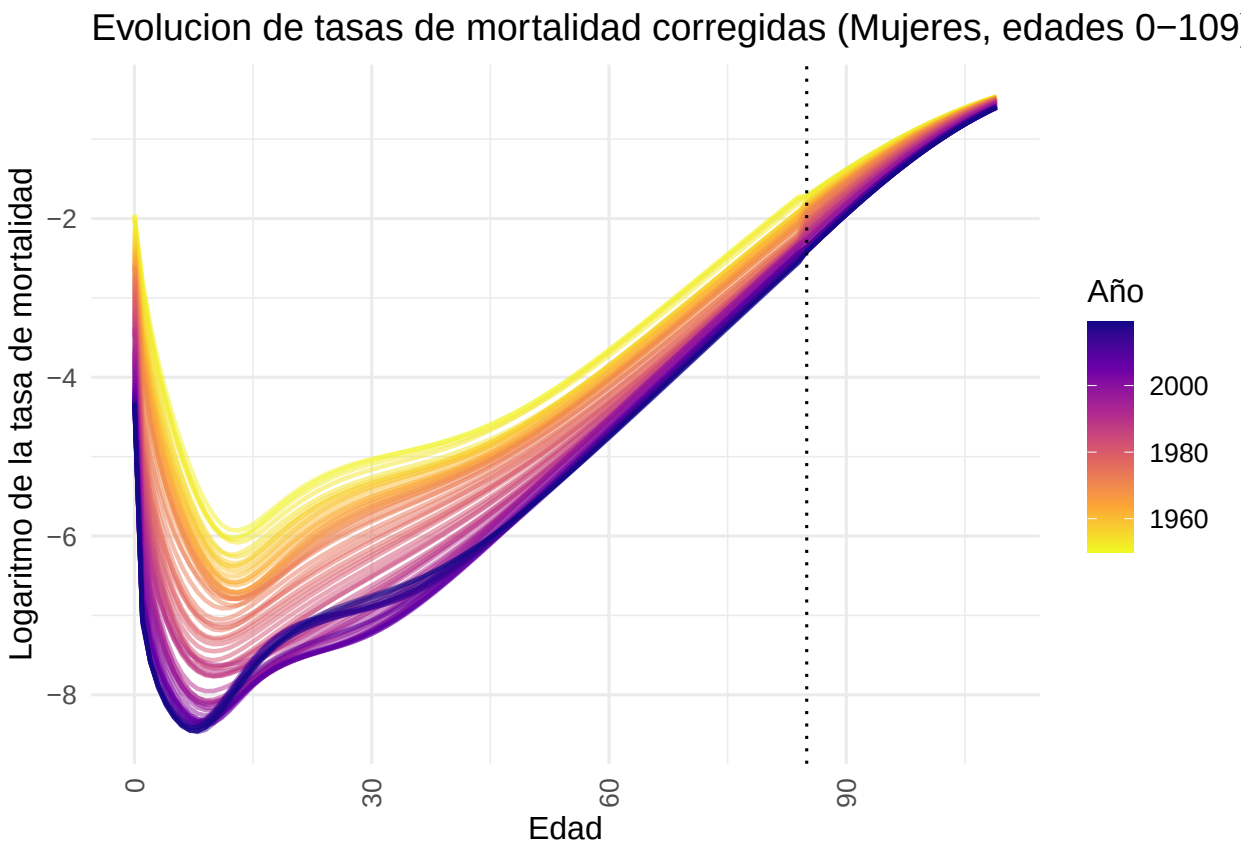
Graficando

```r
ggplot(df_hombres, aes(x = Edad, y = log(Tasa), group = Año, color = Año)) +
  geom_line(alpha = 0.5, linewidth = 0.8) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  theme_minimal(base_size = 12) +
  labs(title = "Evolucion de tasas de mortalidad corregidas (Hombres, edades 0-109)",
       x = "Edad",
       y = "Logaritmo de la tasa de mortalidad",
       color = "Año") +
  theme(legend.position = "right",
        strip.text = element_text(size = 10),
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))+
```

```
  geom_vline(xintercept = 85, linetype="dotted")
```




```
ggplot(df_mujeres, aes(x = Edad, y = log(Tasa), group = Año, color = Año)) +
  geom_line(alpha = 0.5, linewidth = 0.8) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  theme_minimal(base_size = 12) +
  labs(title = "Evolucion de tasas de mortalidad corregidas (Mujeres, edades 0-109)",
       x = "Edad",
       y = "Logaritmo de la tasa de mortalidad",
       color = "Año") +
  theme(legend.position = "right",
        strip.text = element_text(size = 10),
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))+
  geom_vline(xintercept = 85, linetype="dotted")
```



Usando SVD para analizar las TEM

Usaremos descomposición en valores singulares (SVD) para analizar las TEM. Para ello requerimos primero calcular ciertos parámetros. Principalmente, `ln_x_t` que entenderemos como el promedio por file de los logaritmos naturales. Para sesgar nuestro modelo a la estructura reciente de la población utilizaremos las últimas 2 filas (2018, 2019).

```
ln_x_t = log(mxt)
ax = as.data.frame(rowMeans(ln_x_t[, c("2018", "2019")], na.rm = TRUE))
colnames(ax) = "ax"
```

Realizando el andamiaje necesario para las etiquetas del dataframe,

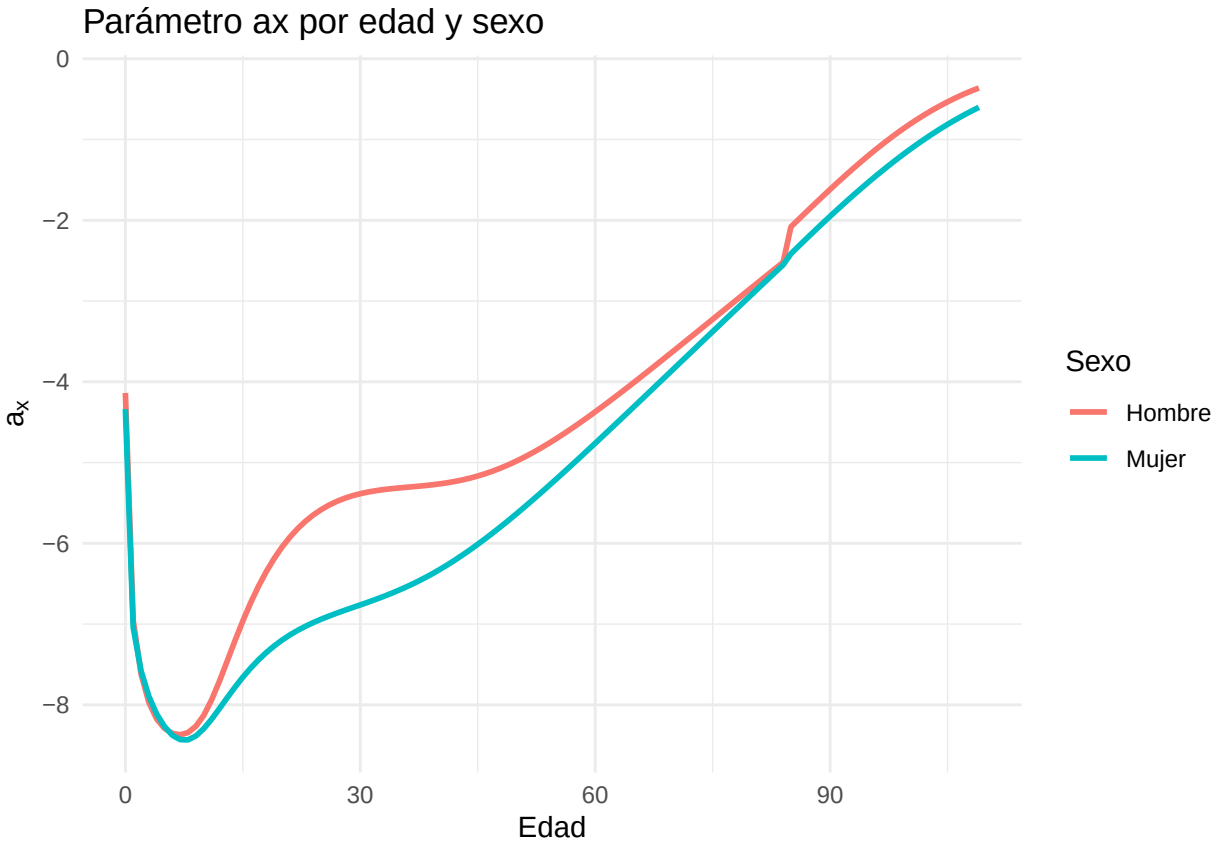
```
ax$Edad <- rep(0:109, 2)
ax$Sexo <- rep(c("Hombre", "Mujer"), each = 110)
```

Graficando

```
ggplot(ax, aes(x = Edad, y = ax, color = Sexo)) +
  geom_line(size = 1) +
  theme_minimal() +
  labs(title = "Parámetro ax por edad y sexo",
       x = "Edad",
       y = expression(a[x]),
       color = "Sexo")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
```

```
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Para aplicar el método de SVD centraremos la matriz

```
ln_x_t_centrada <- ln_x_t %>%
  mutate(across(everything(), ~ .x - ax[,1]))
```

Para poder aplicar la estimación por SVD de `bx` y de `kt`,

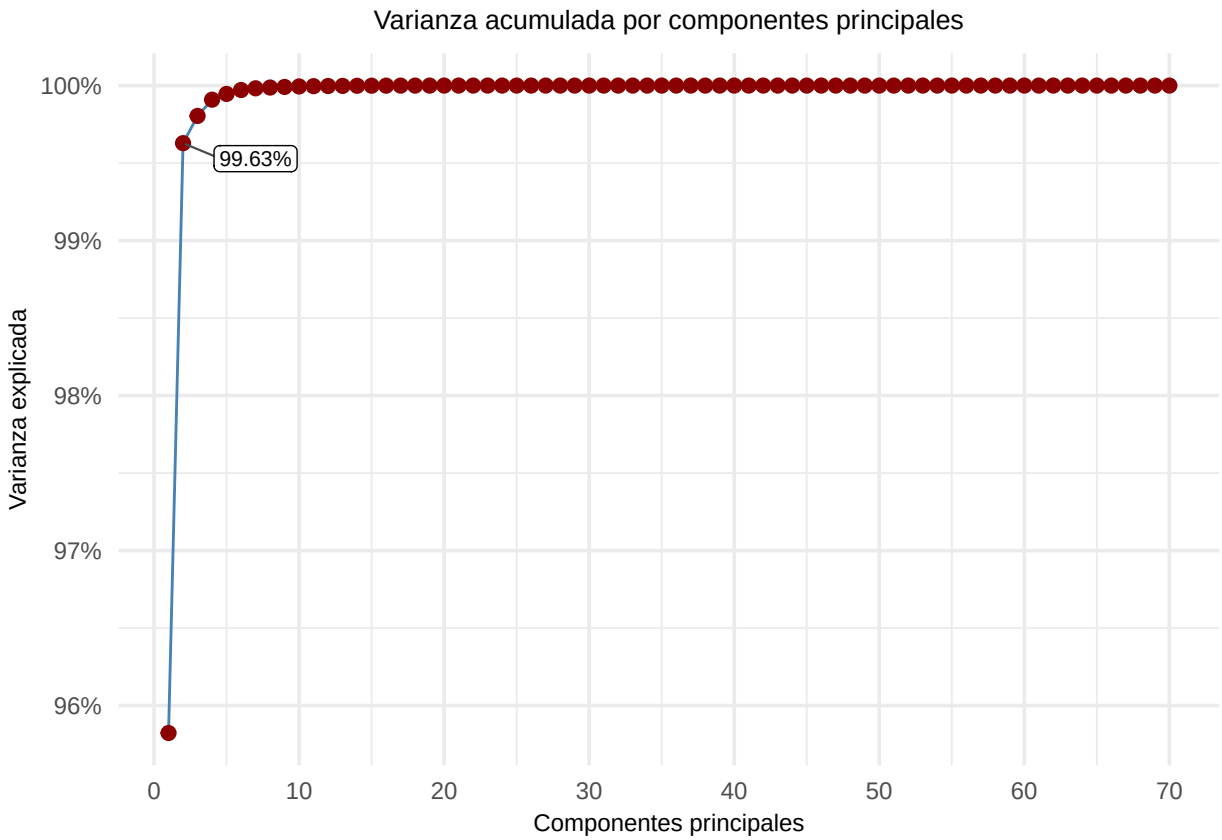
```
svd_resultado <- svd(ln_x_t_centrada, nu =220, nv = 70)
U <- svd_resultado$u   # Edad
D <-  svd_resultado$d  # Valores singulares
V <- (svd_resultado$v) # Tiempo
```

Interpretación

Conociendo ahora los valores singulares asociados, calculemos la varianza explicada. Así conoceremos cuánto se logra capturar

```
var_explicada <- svd_resultado$d^2 / sum(svd_resultado$d^2)
var_explicada_acum <- cumsum(var_explicada)
df <- data.frame(
  Componente = 1:length(var_explicada_acum),
  Varianza = var_explicada_acum)

# Valor del punto 2
punto2 <- df[df$Componente == 2, ]
punto2$label <- paste0(round(punto2$Varianza*100, 2), "%")
```

```r
# Gráfica
ggplot(df, aes(x = Componente, y = Varianza)) +
  geom_line(color = "steelblue", linewidth = 0.5) +
  geom_point(color = "darkred", size = 2) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  scale_x_continuous(breaks = seq(0, max(df$Componente), by = 10)) +
  labs(
    x = "Componentes principales",
    y = "Varianza explicada",
    title = "Varianza acumulada por componentes principales"
  ) +
  geom_label_repel(
    data = punto2,
    aes(label = label),
    fill = "white",
    color = "black",
    size = 2.7,                  # texto más pequeño
    label.size = 0.2,          # grosor del borde del rectángulo
    label.padding = unit(0.14, "lines"),  # padding más chico
    nudge_x = 5,
    nudge_y = -0.001,
    segment.color = "grey30",
    segment.size = 0.4,        # línea más delgada
    min.segment.length = 0
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5,size = 10),   # tamaño del título
    axis.title = element_text(size = 9),   # títulos de los ejes
    axis.text  = element_text(size = 9)     # números en los ejes
  )
```



Vemos así que con tomar 2 valores explicamos el 99.63% de la varianza. Obtenemos los patrones de edad y temporales de cada componente

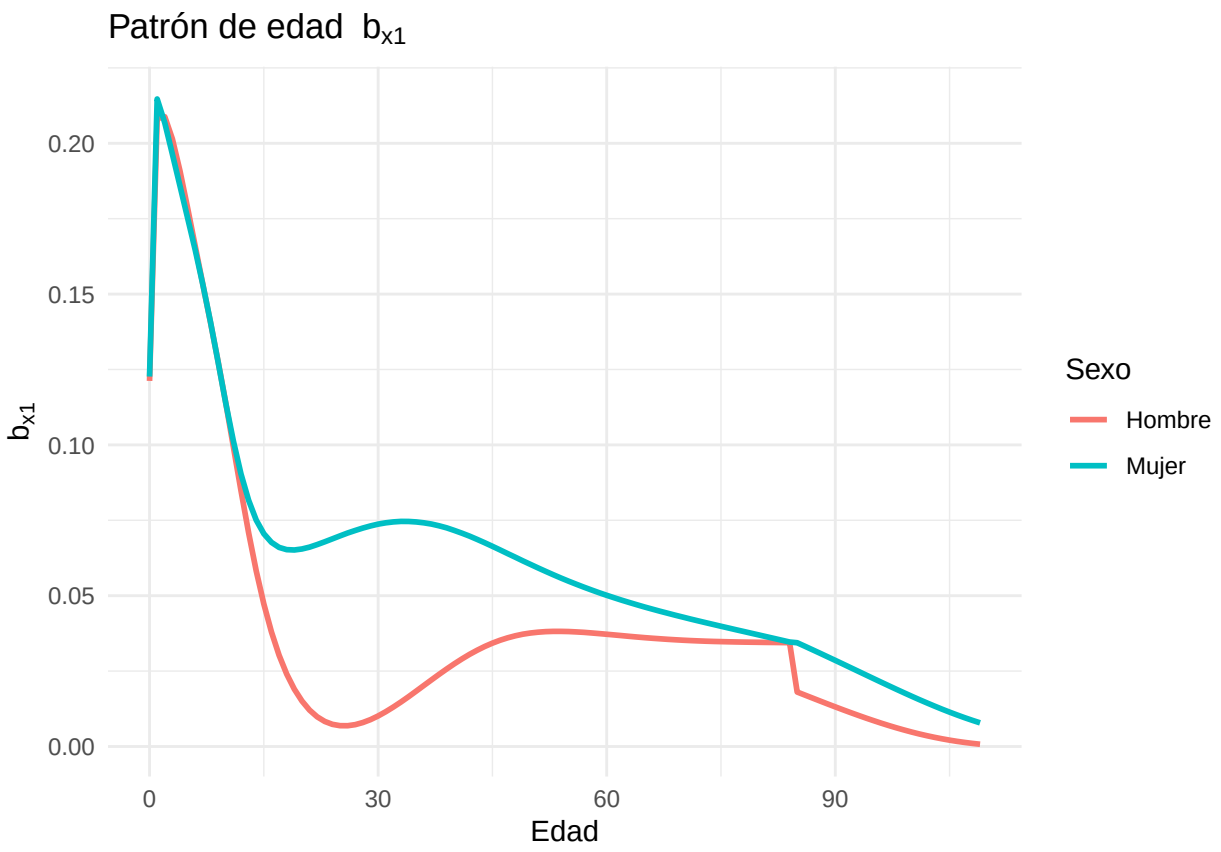
```
# Primer componente
bx1 <- U[, 1]           # patrón de edad 1
kt1 <- D[1] * V[, 1]  # patrón temporal 1
# Segundo componente
bx2 <- U[, 2]           # patrón de edad 2
kt2 <- D[2] *  V[, 2]  # patrón temporal 2
# Realizamos un cambio de signo
bx1 <- -bx1
kt1 <- -kt1
bx2 <- -bx2
kt2 <- -kt2
```

Realizamos el andamiaje para su posterior graficación,

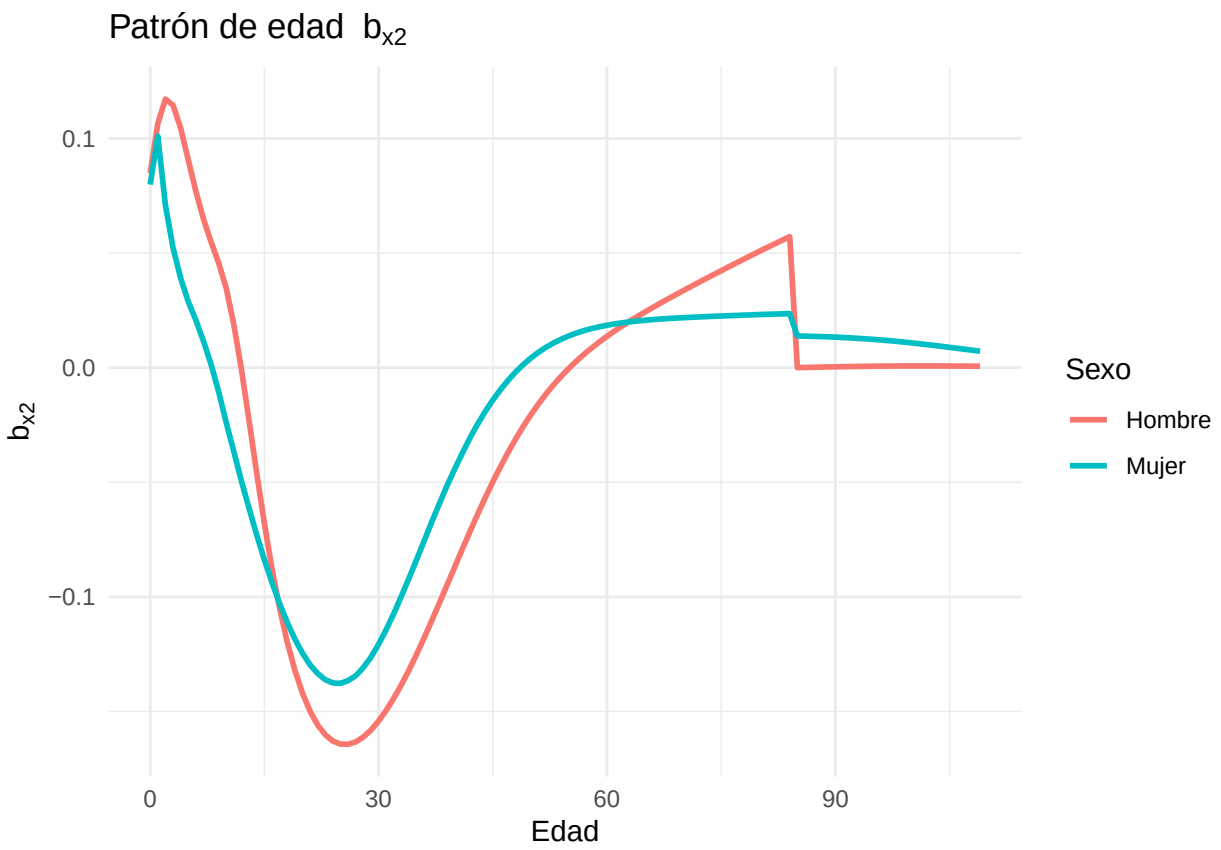
```
df_bx <- data.frame(
  Edad = rep(0:109, 2),
  Sexo = rep(c("Hombre", "Mujer"), each = 110),
  bx1 = bx1,
  bx2 = bx2
)
df_kt1 <- data.frame(
  Año = 1950:2019,
  kt1 = kt1
)
df_kt2 <- data.frame(
  Año = 1950:2019,
```

```
  kt2 = kt2
)
```

```
ggplot(df_bx, aes(x = Edad, y = bx1, color = Sexo)) +
  geom_line(size = 1) +
  theme_minimal() +
  labs(title = expression("Patrón de edad " ~ b[x1]),
       x = "Edad", y = expression(b[x1]), color = "Sexo")
```

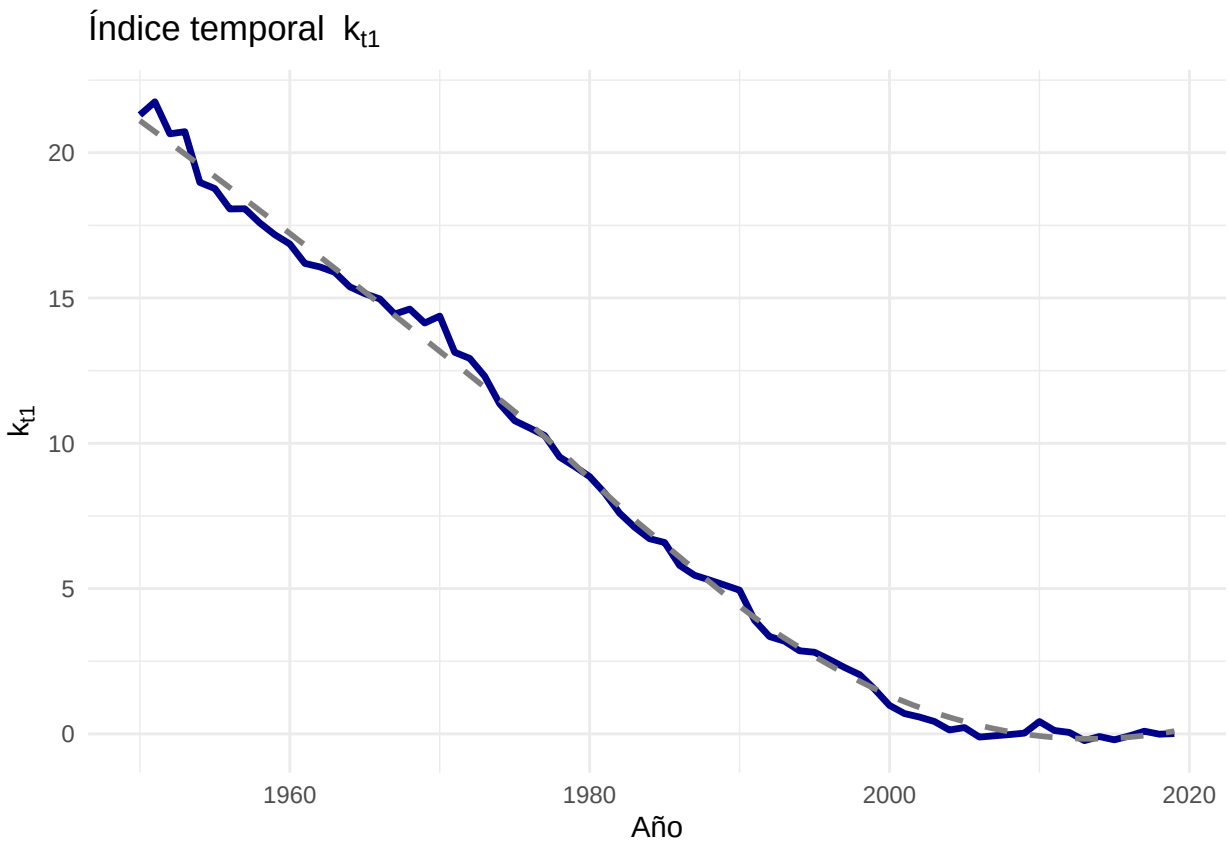



```
ggplot(df_bx, aes(x = Edad, y = bx2, color = Sexo)) +
  geom_line(size = 1) +
  theme_minimal() +
  labs(title = expression("Patrón de edad " ~ b[x2]),
       x = "Edad", y = expression(b[x2]), color = "Sexo")
```



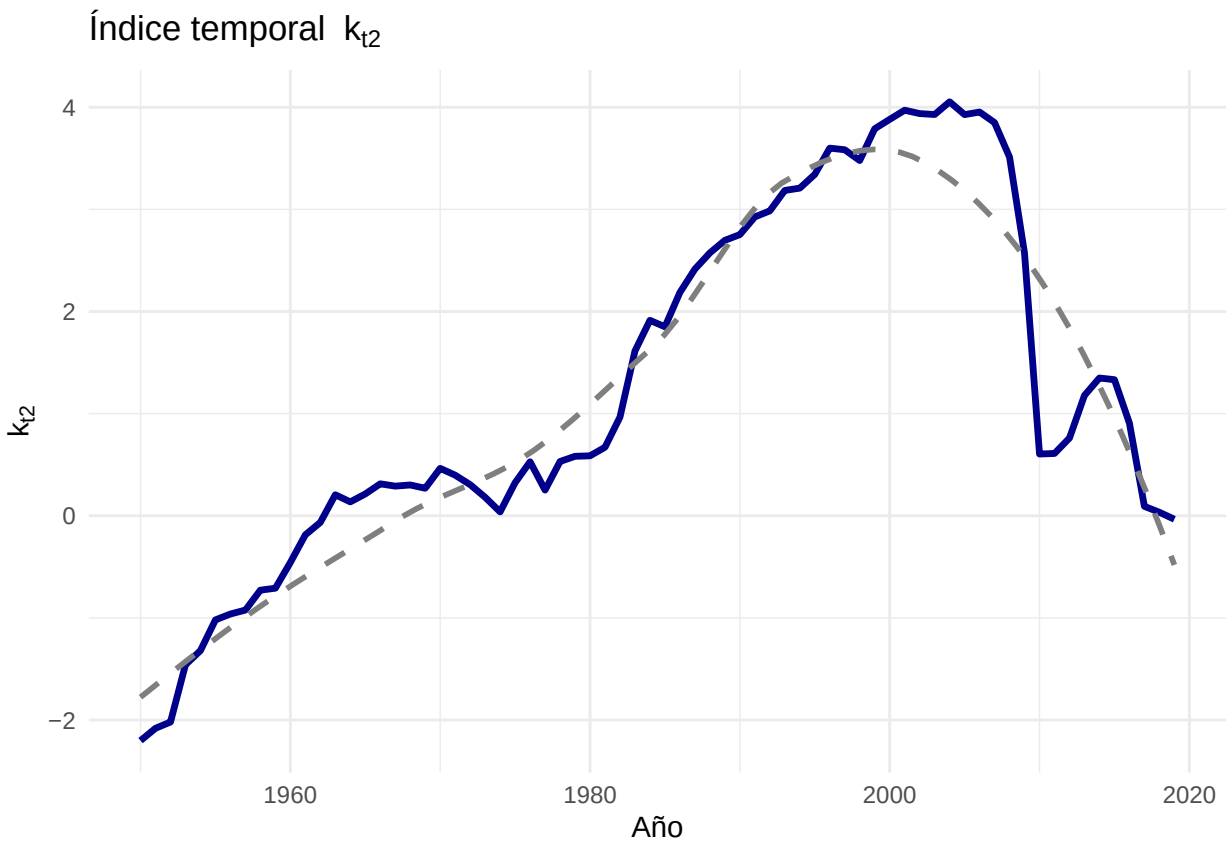

```r
ggplot(df_kt1, aes(x = Año, y = kt1)) +
  geom_line(color = "darkblue", size = 1.2) +
  theme_minimal() +
  labs(
    title = expression("Índice temporal " ~ k[t1]),
    x = "Año",
    y = expression(k[t1])
  ) +
  geom_smooth(method = "loess", se = FALSE, color = "gray50", linetype = "dashed")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```r
ggplot(df_kt2, aes(x = Año, y = kt2)) +
  geom_line(color = "darkblue", size = 1.2) +
  theme_minimal() +
  labs(
    title = expression("Índice temporal " ~ k[t2]),
    x = "Año",
    y = expression(k[t2])
  ) +
  geom_smooth(method = "loess", se = FALSE, color = "gray50", linetype = "dashed")
```

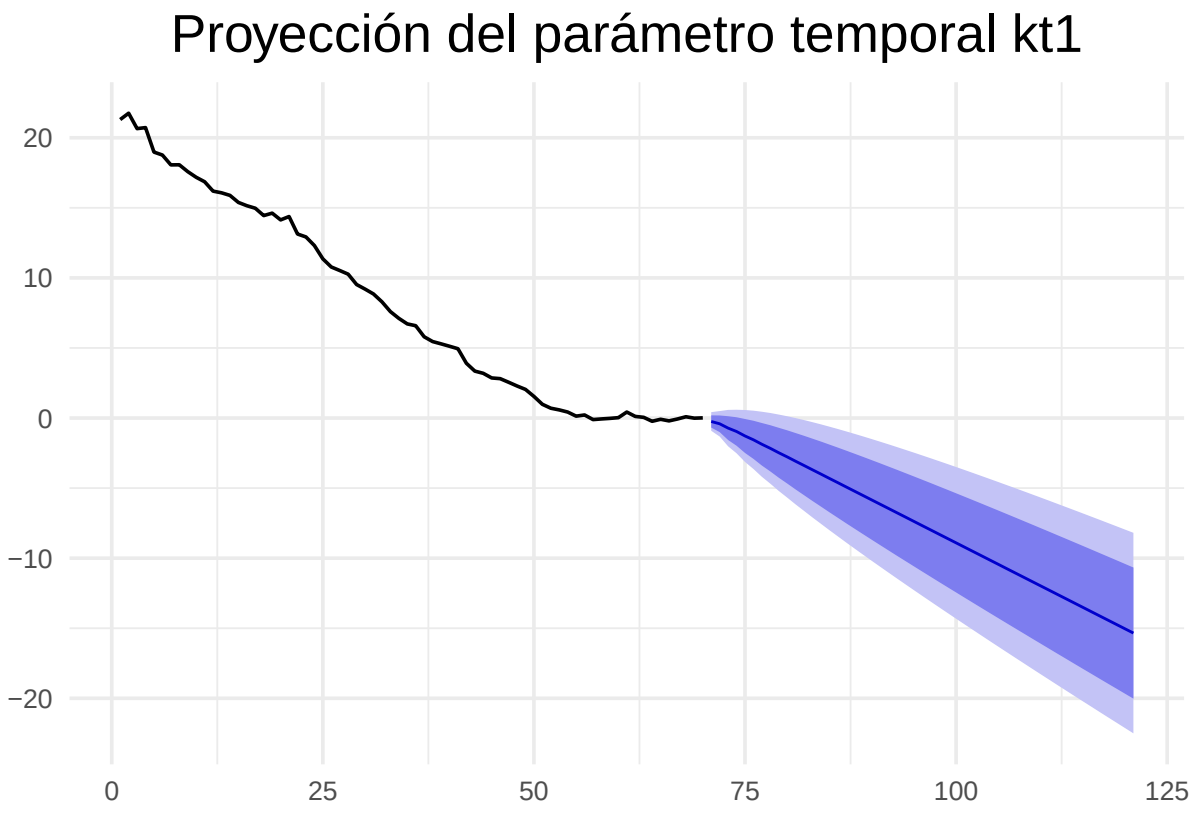
```
## `geom_smooth()` using formula = 'y ~ x'
```



Proyección de componentes kt

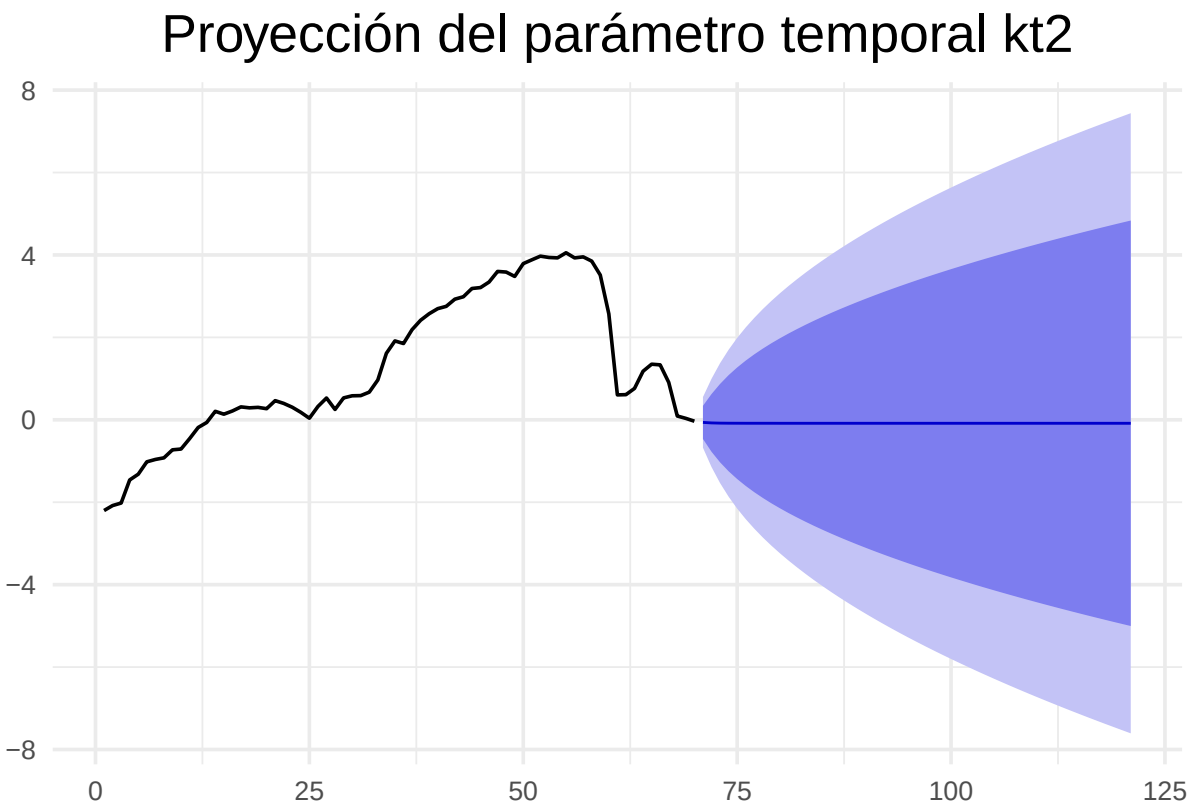
Proyectando usando el modelo Arima. Empezando por kt1

```
modelo_kt1 <- Arima(kt1, order = c(2,1,1), include.drift  = TRUE)
kt1_proy <- forecast(modelo_kt1, h = 51)  # por ejemplo, hasta 2070
# plot(kt1_proy)
autoplot(kt1_proy) +
  labs(title = "Proyección del parámetro temporal kt1", x = "", y = "") +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 20),
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 10))
```



Siguiendo con kt2

```
modelo_kt2 <- Arima(kt2, order = c(1,1,0))
kt2_proy <- forecast(modelo_kt2, h = 51)  # por ejemplo, hasta 2070
#plot(kt2_proy)
autoplot(kt2_proy) +
  labs(title = "Proyección del parámetro temporal kt2", x = "", y = "") +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 20),
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 10))
```



Ahora estudiemos las componentes individuales

```
comp1 <- outer(bx1, kt1_proy$mean, "*")
comp2 <- outer(bx2, kt2_proy$mean, "*")
```

Donde crearemos la matriz de proyecciones de 2020 a 2070.

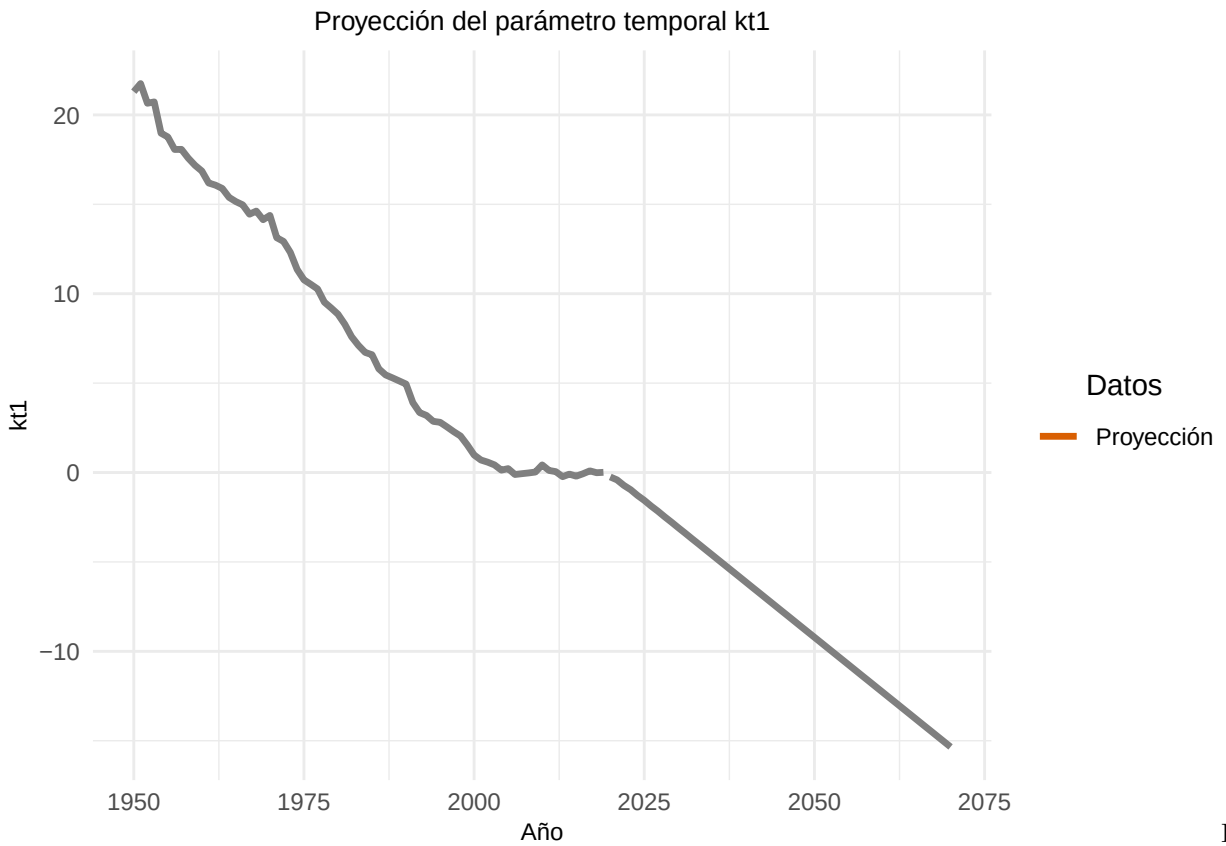
```
a = as.data.frame.matrix(comp1 + comp2 + ax$ax)
mxt_proy = as.data.frame.matrix(exp(comp1 + comp2 + ax$ax))

colnames(mxt_proy) = seq(2020,2070)
rownames(mxt_proy) = seq(1,220)
```

Preparando los gráficos de las proyecciones de kt1

```
años_proy <- 2020:(2019 + length(kt1_proy$mean))
kt1_proy_df <- data.frame(
  año = 2020:(2019 + length(kt1_proy$mean)),
  valor = kt1_proy$mean,
  tipo = "Proyeccion"
)
kt1_conc_df <- data.frame(
  año = 1950:2019,
  valor = kt1,
  tipo = "Conciliacion"
)
kt1_df <- rbind(kt1_conc_df, kt1_proy_df)

renglon_2019 <- data.frame(año = tail(kt1_conc_df[[1]], 1),valor = tail(kt1_conc_df[[2]], 1), tipo = "P
kt1_proy_df <- rbind(renglon_2019, kt1_proy_df)
kt1_df <- rbind(kt1_conc_df, kt1_proy_df)
```

Graficándolo,

```
ggplot(kt1_df, aes(x = año, y = valor, color = tipo)) +
  geom_line(size = 1.2) +
  labs(title = "Proyección del parámetro temporal kt1", x = "Año", y = "kt1", color = "Datos") +
  scale_color_manual(values = c("Conciliación" = "#1b9e77", "Proyección" = "#d95f02")) +
  theme_minimal() +
  theme(
    legend.title = element_text(hjust = 0.5),
    plot.title = element_text(hjust = 0.5, size = 10),
    axis.title = element_text(size = 9),
    axis.text = element_text(size = 9)
  )
```



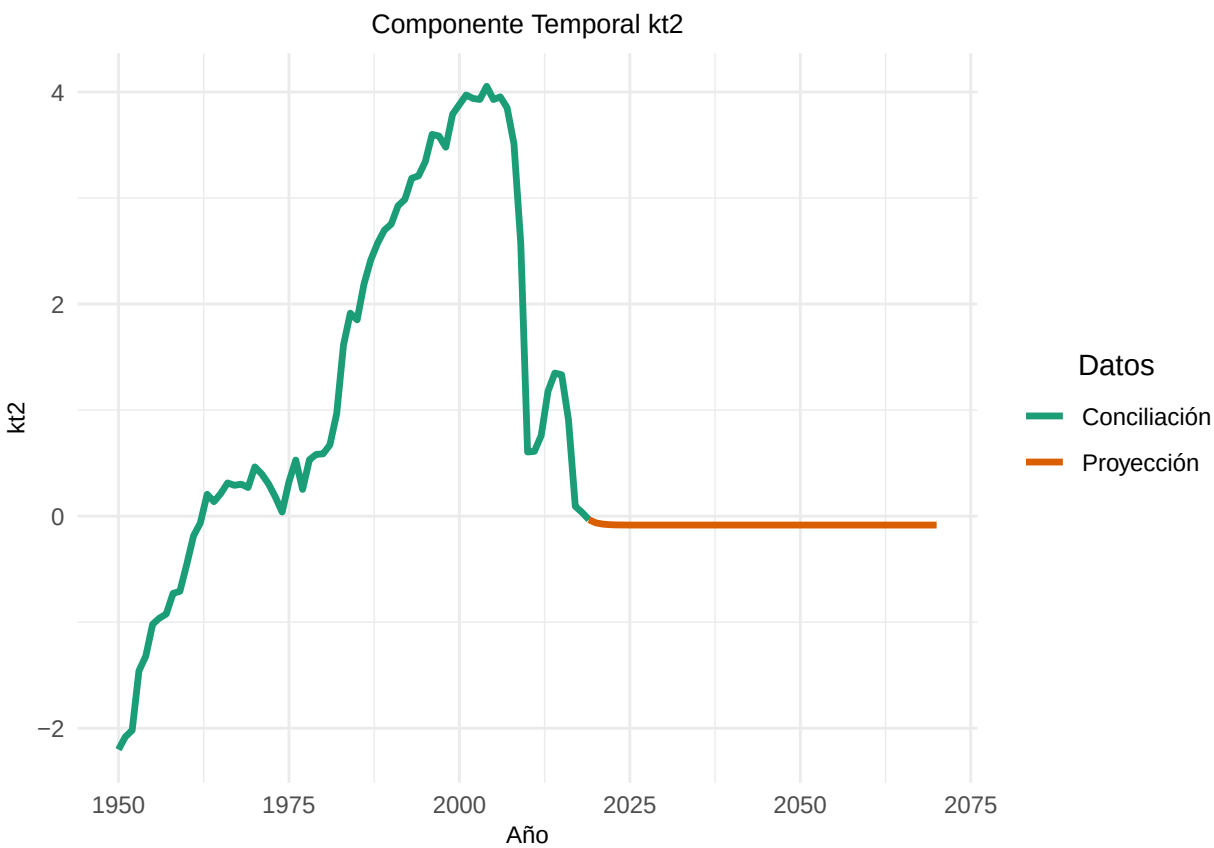
Realizando el proceso análogo para kt2

```
años_proy <- 2020:(2019 + length(kt2_proy$mean))
kt2_proy_df <- data.frame(
  año = 2020:(2019 + length(kt2_proy$mean)),
  valor = kt2_proy$mean,
  tipo = "Proyección"
)

kt2_conc_df <- data.frame(
  año = 1950:2019,
  valor = kt2,
  tipo = "Conciliación"
)
```

```
kt2_df <- rbind(kt2_conc_df, kt2_proy_df)

renglon_2019 <- data.frame(año = tail(kt2_conc_df[[1]], 1),valor = tail(kt2_conc_df[[2]], 1), tipo = "P
kt2_proy_df <- rbind(renglon_2019, kt2_proy_df)
kt2_df <- rbind(kt2_conc_df, kt2_proy_df)
```

Graficándolo

```
ggplot(kt2_df, aes(x = año, y = valor, color = tipo)) +
  geom_line(size = 1.2) +
  labs(title = "Componente Temporal kt2", x = "Año", y = "kt2", color = "Datos") +
  scale_color_manual(values = c("Conciliación" = "#1b9e77", "Proyección" = "#d95f02")) +
  theme_minimal()+
  theme(
    legend.title = element_text(hjust = 0.5),
    plot.title = element_text(hjust = 0.5, size = 10),
    axis.title = element_text(size = 9),
    axis.text = element_text(size = 9)
  )
```

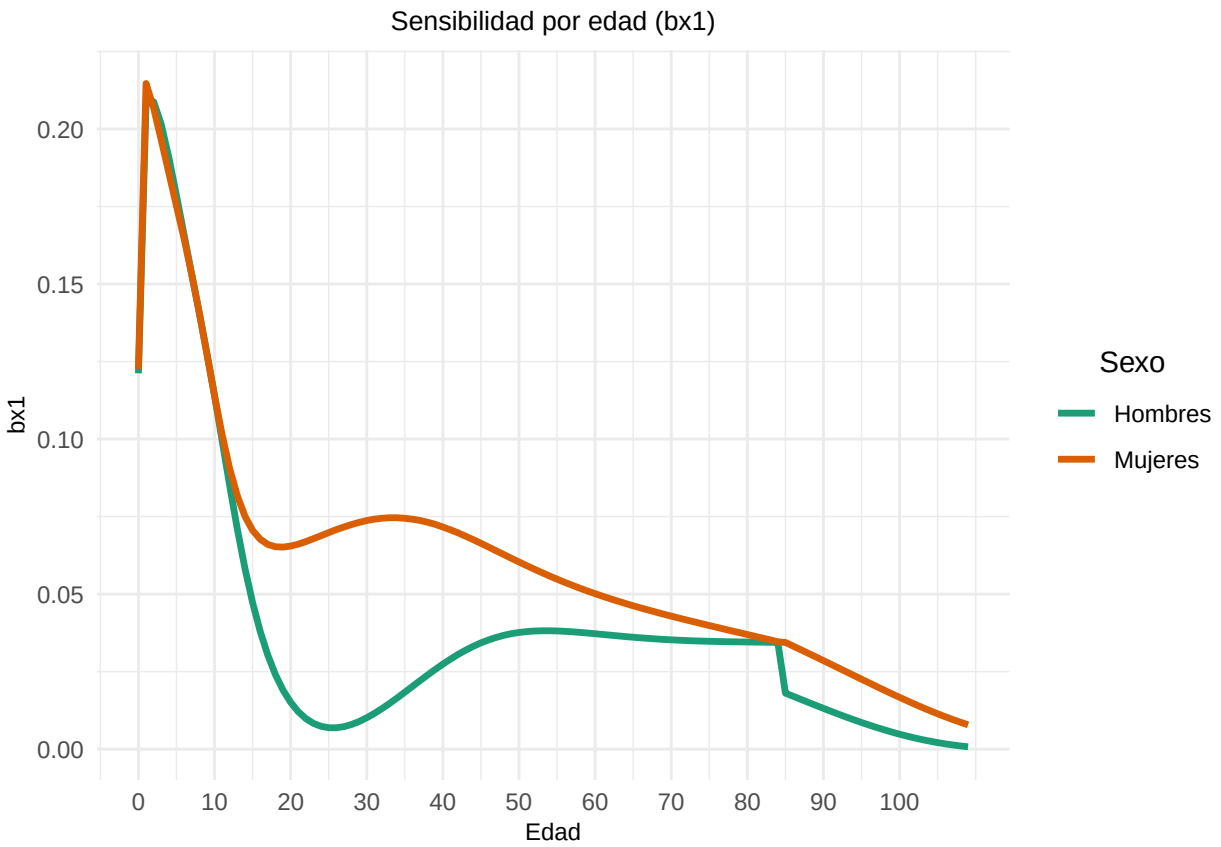


Por último realizando el andamiaje para bx1

```
bx1_df <- data.frame(
  edad = rep(0:109, 2),
  sexo = rep(c("Hombres", "Mujeres"), each = 110),
  sensibilidad = bx1
)
```

Graficando

```r
ggplot(bx1_df, aes(x = edad, y = sensibilidad, color = sexo)) +
  geom_line(size = 1.2) +
  labs(title = "Sensibilidad por edad (bx1)",
       x = "Edad", y = "bx1", color = "Sexo") +
  scale_color_manual(values = c("Hombres" = "#1b9e77", "Mujeres" = "#d95f02")) +
  scale_x_continuous(breaks = seq(0, max(bx1_df$edad), by = 10)) +
  theme_minimal()+
  theme(
    legend.title = element_text(hjust = 0.5),
    plot.title = element_text(hjust = 0.5, size = 10),
    axis.title = element_text(size = 9),
    axis.text = element_text(size = 9)
  )
```



Ahora vamos a extender estas proyecciones a la población. Para ello necesitamos crear las filas necesarias.

```r
edad <- rep(0:109, 2)
sexo <- rep(c("Hombres", "Mujeres"), each = 110)
mxt = as.data.frame.matrix(mxt)
```

Nombramos a la conciliación

```r
mxt_conc_largo <- mxt %>%
  mutate(edad = edad, sexo = sexo) %>%
  pivot_longer(cols = -c(edad, sexo), names_to = "año", values_to = "mxt") %>%
  mutate(año = as.numeric(sub("X", "", año)), tipo = "Conciliación")
```

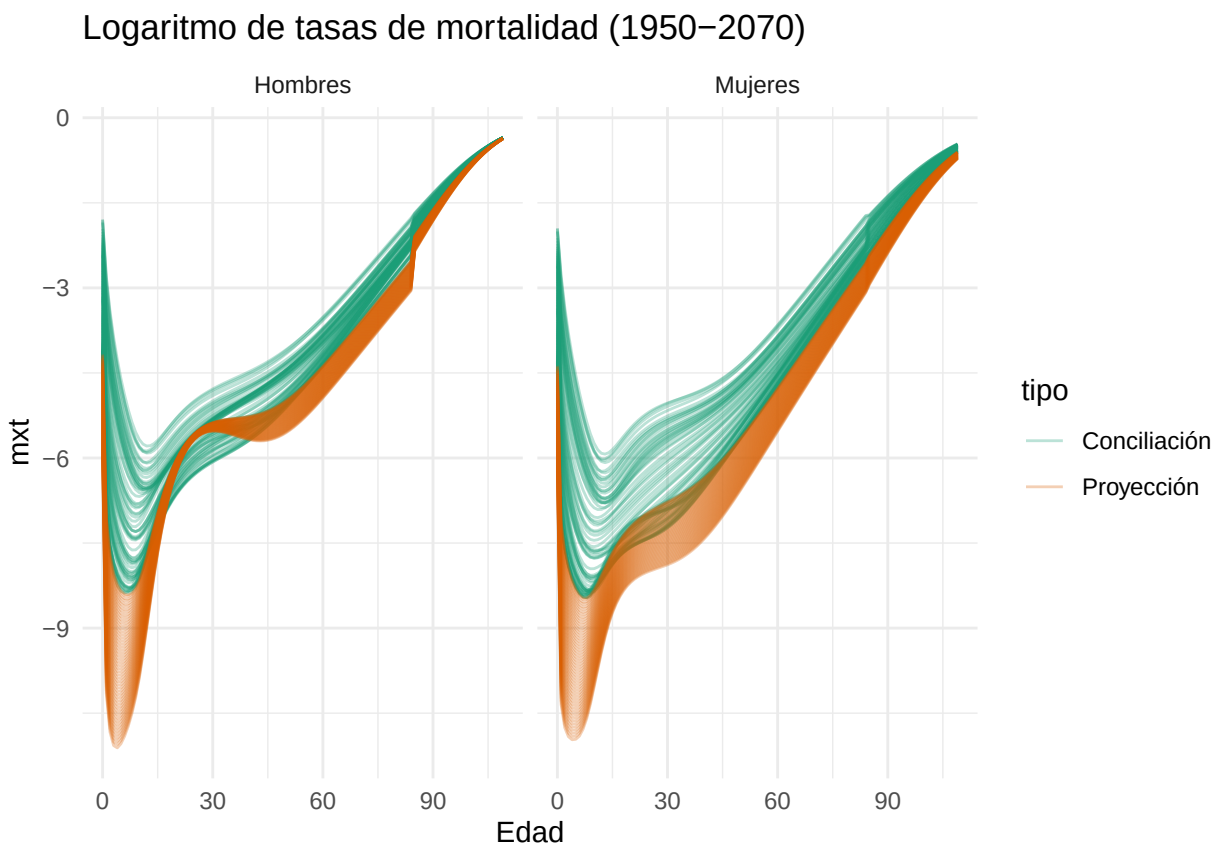
Y de igual manera le damos un nombre a la proyección

```r
mxt_proy_largo <- mxt_proy %>%
  mutate(edad = edad, sexo = sexo) %>%
  pivot_longer(cols = -c(edad, sexo), names_to = "año", values_to = "mxt") %>%
  mutate(año = as.numeric(gsub("[^0-9]", "", año)), tipo = "Proyección")
```

Que unimos para tener la población conciliada de 1950 a 2019 y proyectada de 2020 a 2050

```r
mxt_total <- bind_rows(mxt_conc_largo, mxt_proy_largo)
```

```r
ggplot(mxt_total, aes(x = edad, y = log(mxt), group = año, color = tipo)) +
  geom_line(alpha = 0.3) +
  facet_wrap(~sexo) +
  labs(title = "Logaritmo de tasas de mortalidad (1950-2070)",
       x = "Edad", y = "mxt") +
  scale_color_manual(values = c("Conciliación" = "#1b9e77", "Proyección" = "#d95f02")) +
  theme_minimal()
```



Juntaremos el total en un dataframe

```r
df_largo = mxt_total
```

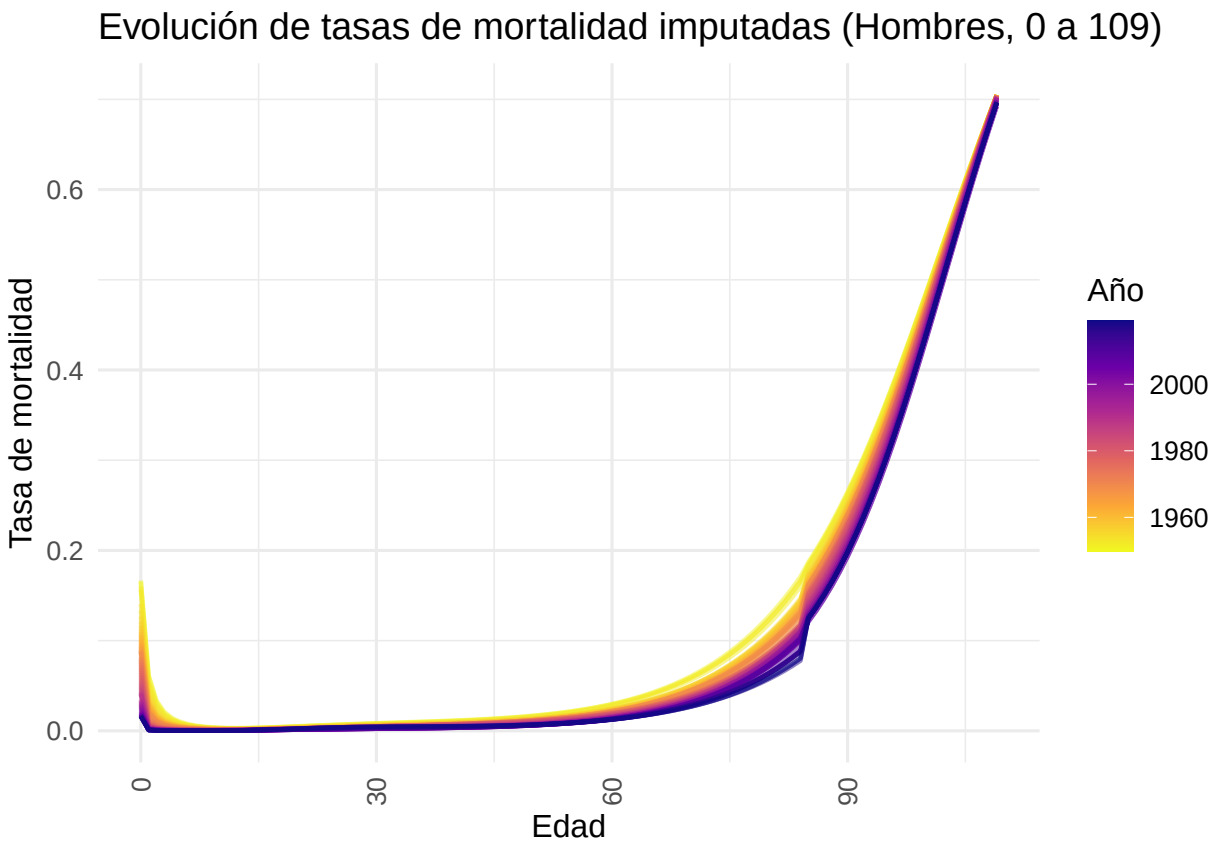
Detallando para hombres

```r
df_hombres <- df_largo %>%
  filter(sexo == "Hombres", año >= 1950, año <= 2019)
```

Graficamos

```r
ggplot(df_hombres, aes(x = edad, y = mxt, group = año, color = año)) +
  geom_line(alpha = 0.5, linewidth = 0.8) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
```

```
  theme_minimal(base_size = 12) +
  labs(title = "Evolución de tasas de mortalidad imputadas (Hombres, 0 a 109)",
       x = "Edad",
       y = "Tasa de mortalidad",
       color = "Año") +
  theme(legend.position = "right",
        strip.text = element_text(size = 10),
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```

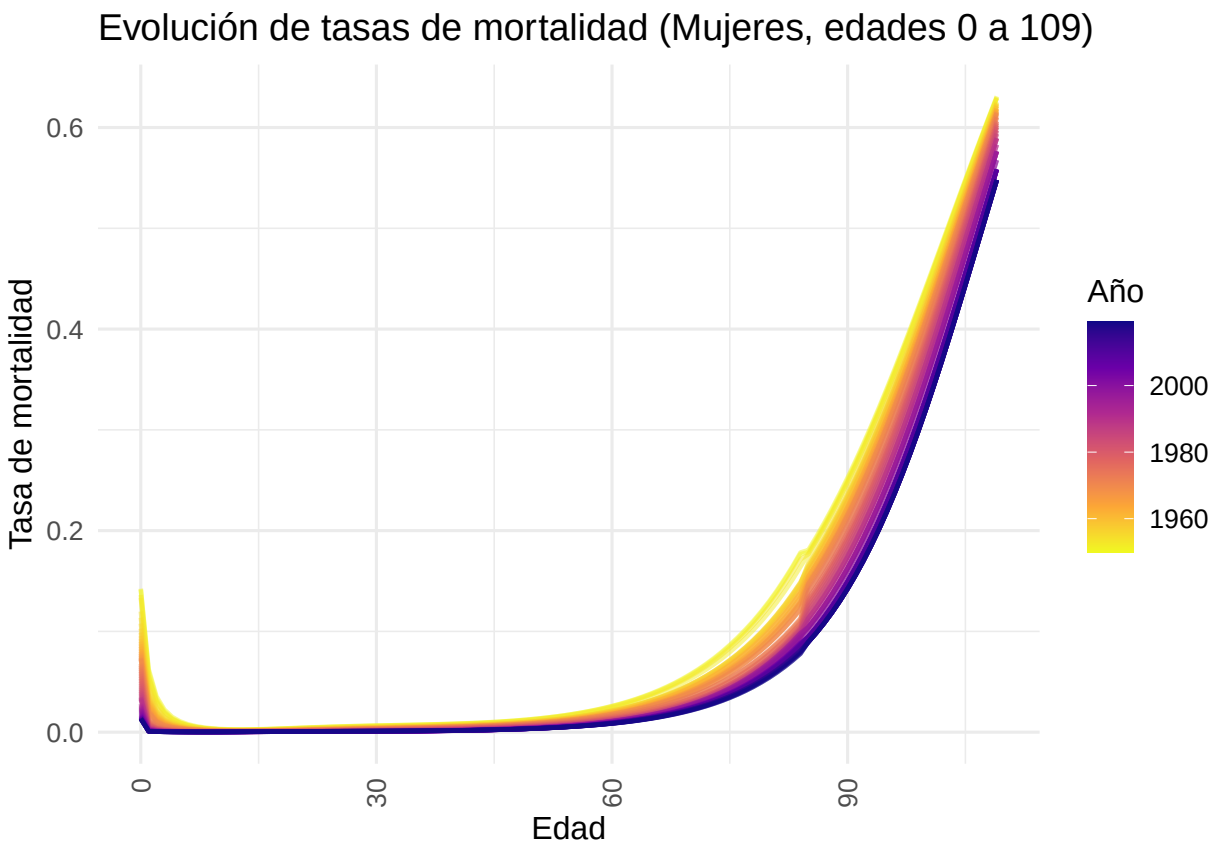


Haciendo énfasis en la gráfica de las mujeres

```
df_mujeres <- df_largo %>%
  filter(sexo == "Mujeres", año >= 1950, año <= 2019)
```

Graficamos

```
ggplot(df_mujeres, aes(x = edad, y = mxt, group = año, color = año)) +
  geom_line(alpha = 0.5, linewidth = 0.8) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  theme_minimal(base_size = 12) +
  labs(title = "Evolución de tasas de mortalidad (Mujeres, edades 0 a 109)",
       x = "Edad",
       y = "Tasa de mortalidad",
       color = "Año") +
  theme(legend.position = "right",
        strip.text = element_text(size = 10),
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```



Tasa de mortalidad

Tablas de vida

Definiremos una función de la tabla de vida que tendrá un radix de $l_0 = 10^5$

```
tabla_vida <- function(df, radix = 100000, n = 1)
{
  calc_tabla <- function(subdf) {
    subdf <- subdf %>% arrange(edad)
    mx <- subdf$mxt
    w <- length(mx)
    # n_ax
    nax <- rep(n/2, w)
    # qx
    qx <- (n * mx) / (1 + (n - nax) * mx)
    qx[w] <- 1
    # px
    px <- 1 - qx
    # lx
    lx <- numeric(w)
    lx[1] <- radix
    for (i in 1:(w - 1))
    {
      lx[i + 1] <- lx[i] * px[i]
    }
    # dx
```

```r
    dx <- lx * qx
    # Lx
    Lx <- numeric(w)
    for (i in 1:(w - 1))
    {
      Lx[i] <- n * lx[i + 1] + nax[i] * dx[i]
    }
    Lx[w] <- nax[w] * dx[w]
    # Tx
    Tx <- rev(cumsum(rev(Lx)))
    # ex
    ex <- Tx / lx
    subdf %>%
      mutate(
        qx = qx,
        px = px,
        lx = lx,
        dx = dx,
        Lx = Lx,
        Tx = Tx,
        ex = ex
      )
  }
  df %>%
    group_by(sexo, año, tipo) %>%
    group_modify(~ calc_tabla(.x)) %>%
    ungroup()
}
```

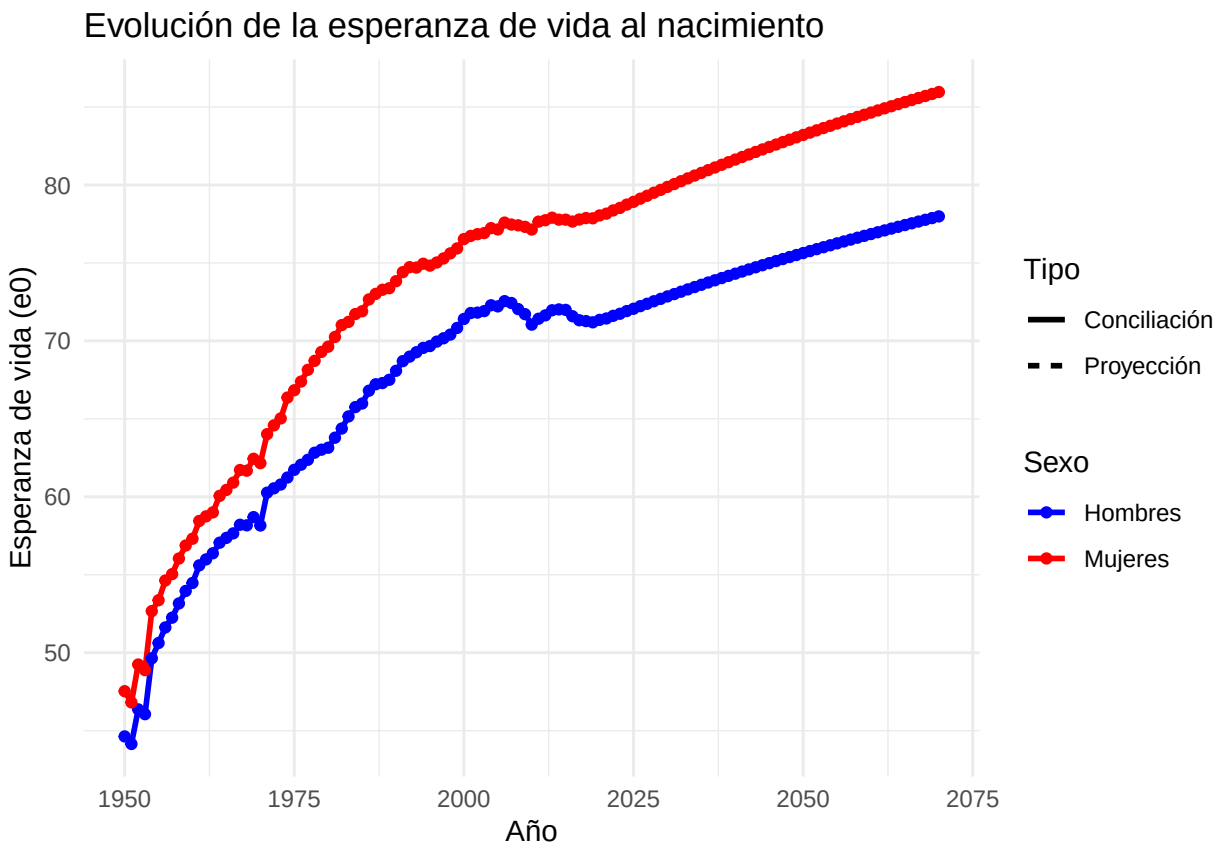
Es así con que basta llamar a nuestras funciones

```r
resultado <- tabla_vida(mxt_total)
```

Generamos elun filtro para distinguir a la mortalidad infantil

```r
df_e0 <- resultado %>%
  filter(edad == 0)
```

```r
ggplot(df_e0, aes(x = año, y = ex, color = sexo, linetype = tipo)) +
  geom_line(size = 1) +
  geom_point() +
  theme_minimal() +
  labs(
    title = "Evolución de la esperanza de vida al nacimiento",
    x = "Año",
    y = "Esperanza de vida (e0)",
    color = "Sexo",
    linetype = "Tipo"
  ) +
  scale_color_manual(values = c("Hombres" = "blue", "Mujeres" = "red"))
```




```r
ggplot(resultado, aes(x = edad, y = ex, color = sexo)) +
  geom_line(alpha = 0.8) +
  facet_wrap(~año) +
  theme_minimal() +
  labs(
    title = "Esperanza de vida por edad, sexo y año",
    x = "Edad",
    y = "Esperanza de vida (e0)",
    color = "Sexo",
    linetype = "Tipo"
  ) +
  scale_color_manual(values = c("Hombres" = "blue", "Mujeres" = "red"))
```

```
## Ignoring unknown labels:
## * linetype : "Tipo"
```

Relaciones de sobrevivencia

Con base en esta última función generamos el cálculo desde el primer grupo de edad hasta el último y esto nos permite graficarlo.

```
relaciones_sobrevivencia <- function(df_tidy, n = 1) {
  df_tidy %>%
    arrange(sexo, año, edad) %>%
    group_by(sexo, año) %>%
    mutate(
      # S_x = l_{x+n} / l_x
      Sx = lead(lx, n) / lx,
      # Relación desde el nacimiento
      S0x = lx / first(lx)
    ) %>%
    ungroup()
}
```

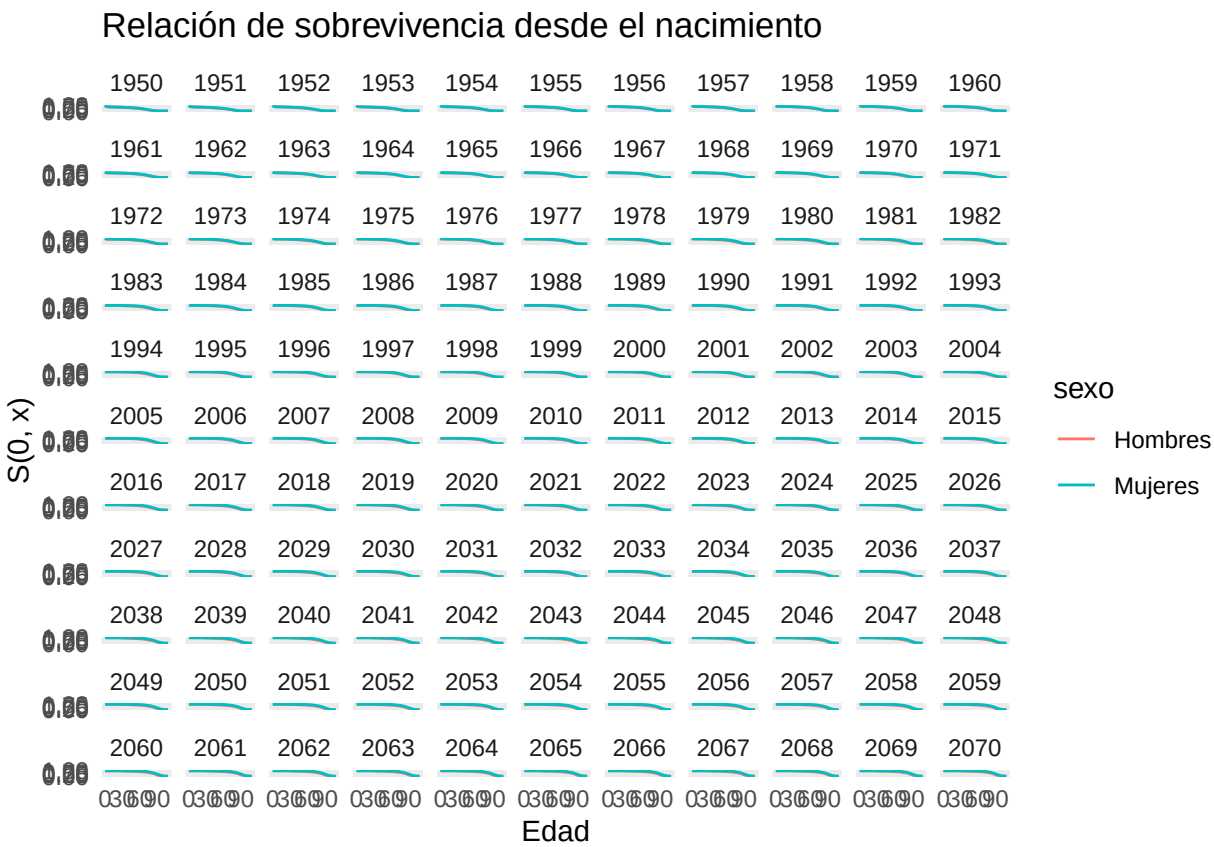
Aplicándolo a nuestros datos

```
df_surv <- relaciones_sobrevivencia(resultado)
```

Graficando la sobrevivencia acumulada

```
ggplot(df_surv, aes(x = edad, y = S0x, color = sexo)) +
  geom_line() +
  facet_wrap(~año) +
  theme_minimal() +
  labs(
```

```
    title = "Relación de sobrevivencia desde el nacimiento",
    y = "S(0, x)",
    x = "Edad"
  )
```



Y la de pasar al siguiente grupo de edad

```
ggplot(df_surv, aes(x = edad, y = Sx, color = sexo)) +
  geom_line() +
  facet_wrap(~año) +
  theme_minimal() +
  labs(
    title = "Relación de sobrevivencia por intervalo",
    y = "S(x, x+n)",
    x = "Edad"
  )
```

```
## Warning: Removed 242 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

Relación de sobrevivencia por intervalo
S(x, x+n)
Edad
sexo
Hombres
Mujeres